

ArqSysIA - Foundational Document

Documento Fundacional del Proyecto

Versión: 1.0

Fecha de Creación: 3 de Octubre, 2025

Autor: Desarrollador de Software IA

Ubicación: Crespo, Entre Ríos, Argentina



Visión y Propósito

Declaración de Visión

ArqSysIA (Arquitecto de Sistemas con Inteligencia Artificial) es una aplicación de escritorio inteligente diseñada para asistir al desarrollador en el **diseño y definición de arquitecturas de aplicaciones de software** que debe desarrollar para sus clientes.

Objetivo Principal

Automatizar y optimizar el proceso de arquitectura de software, reduciendo significativamente el tiempo y esfuerzo necesarios para:

- Analizar y documentar requerimientos
- Diseñar arquitecturas de sistemas
- Definir estructuras de archivos y componentes
- Identificar dependencias tecnológicas
- Generar documentación técnica (MVP y roadmap completo)

Propuesta de Valor

- Ahorro de tiempo:** Automatización del análisis arquitectural inicial
- Guía de desarrollo:** Roadmap claro para implementación
- Asistente de debugging:** Validación de diseños antes de codificar
- Consultor virtual:** Razonamiento experto disponible 24/7

Contexto del Usuario

Perfil

- **Rol:** Desarrollador de Software IA profesional
- **Modelo de Trabajo:** Workstation potente individual (no desarrollo en equipo por el momento)
- **Necesidad:** Herramienta personal para arquitectura de sistemas para clientes
- **Filosofía:** Prefiere control y simplicidad sobre complejidad innecesaria

Infraestructura Disponible

Hardware de Desarrollo:

- **CPU:** AMD Ryzen 9 7900 (12 núcleos, 24 hilos)
- **RAM:** 128 GB DDR5
- **Almacenamiento:** 2 TB NVMe (2x 1TB unificados)
- **Motherboard:** B650 AORUS ELITE AX V2
- **GPU:** Gigabyte GeForce RTX 5070 Ti
 - Arquitectura: NVIDIA Blackwell
 - VRAM: 16 GB GDDR7
 - CUDA Cores: 8,960
 - Tensor Cores: 280 (5ta generación)
 - Bus: 256-bit
 - Ancho de banda: 896 GB/s
 - TDP: 300W

Software Base:

- **OS:** Linux (distribución por confirmar)
- **Infraestructura IA:** Ollama (ya instalado localmente con modelos existentes)
- **Contenedores:** Docker (para aislamiento del proyecto)

Decisiones Arquitecturales Fundamentales

1. Enfoque Multi-Modelo Especializado

Decisión: Utilizar un ensamble de 3 modelos LLM especializados, cada uno optimizado para una fase específica del pipeline.

Modelos Seleccionados:

Fase	Modelo	Tamaño	Propósito	Justificación
Análisis	DeepSeek-R1-Distill-Qwen-32B	32B	Razonamiento arquitectural profundo	Entrenado con RL para razonamiento, genera chain-of-thought naturalmente, performance comparable a o1
Generación	Qwen2.5-Coder-32B	32B	Generación de código y estructura	Especializado en código multi-lenguaje, comprensión de arquitecturas, más rápido que R1
Validación	DeepSeek-R1-14B	14B	Verificación y debugging	Mantiene capacidades de razonamiento, más rápido (~2x), menor consumo VRAM

Ventajas del Enfoque:

- Cada modelo optimizado para su tarea específica
- Balance entre calidad y velocidad
- Uso eficiente de los 16GB VRAM disponibles
- Licencias permisivas (MIT, Apache 2.0)

2. Arquitectura de Software: Pipeline Tradicional vs Agentes

Decisión: Implementar una **arquitectura de pipeline tradicional con orquestador**, NO un sistema multi-agente.

Razones de la Decisión:

A favor del pipeline tradicional:

1. **Flujo lineal y predecible:** Análisis → Generación → Validación
2. **Control explícito:** Visibilidad total sobre cada paso del proceso
3. **Debugging simplificado:** Más fácil identificar y corregir errores
4. **Sin overhead innecesario:** No hay framework de agentes agregando complejidad
5. **Checkpoints manuales:** Usuario puede revisar y aprobar entre fases
6. **Simplicidad de mantenimiento:** Código más limpio y directo
7. **No requiere alta concurrencia:** Aplicación personal, no servicio multi-usuario

En contra de agentes (para este caso):

1. Frameworks de agentes agregan complejidad innecesaria
2. Problemas de logging y debugging en frameworks como CrewAI

3. Overhead de coordinación no justificado para flujo lineal
4. Mayor curva de aprendizaje sin beneficio claro
5. El flujo NO requiere autonomía extrema ni toma de decisiones dinámica compleja

Posibilidad futura: La arquitectura está diseñada para evolucionar a agentes (CrewAI recomendado) si en el futuro se requiere:

- Iteración autónoma sin intervención humana
- Múltiples estrategias de diseño en paralelo
- Workflows con muchos branches condicionales

3. Infraestructura: Arquitectura Híbrida

Decisión: Usar Ollama local existente desde contenedor Docker (Opción C - Híbrida).

Configuración:

```
Docker Container (ArqSysIA App)
  ↓ HTTP
Host System (Ollama Server: localhost:11434)
  ├── Modelo Existente (previo)
  ├── DeepSeek-R1-32B (nuevo)
  ├── Qwen2.5-Coder-32B (nuevo)
  └── DeepSeek-R1-14B (nuevo)
```

Ventajas:

-  Aislamiento de aplicación en Docker
-  No duplica modelos (ahorro ~40-60GB por modelo)
-  Aprovecha instalación Ollama existente
-  Uso eficiente de 128GB RAM
-  Fácil migración futura si se necesita separación

Alternativas consideradas y rechazadas:

- **Ollama en Docker dedicado:** Rechazado por duplicación de modelos
- **vLLM:** Reservado para migración futura si se requiere más throughput

Arquitectura del Sistema

Pipeline de Tres Fases

ArqSysIA Core

Pipeline Orchestrator

- Coordina fases
- Gestiona estado global
- Checkpoints manuales

FASE 1:

Analyzer

- Reqs F/NF

- Arquitectura

- Stack Tech

- Componentes

- Dependencias

- MVP Scope

- Roadmap

DeepSeek-R1

32B

FASE 2:

CodeGen

- Files

- Code

- Docs

- Setup

Qwen2.5

Coder

FASE 3:

Validator

- Errors

- Mejoras

- Security

- Missing

DeepSeek-R1

14B

Flujo de Ejecución

1. **Input:** Requerimientos del sistema (siguiendo técnicas de Ingeniería de Requisitos)

2. **Fase 1 - Análisis (DeepSeek-R1-32B):**

- Identifica requerimientos funcionales y no funcionales
- Propone patrón arquitectural justificado
- Define stack tecnológico (prioriza open-source)
- Lista componentes principales
- Estructura de directorios
- Dependencias detalladas
- Alcance MVP
- Roadmap del producto completo
- **Checkpoint:** Usuario revisa antes de continuar

3. **Fase 2 - Generación (Qwen2.5-Coder-32B):**

- Genera estructura completa de archivos
- Código inicial de archivos clave
- Scripts de setup/deployment
- Documentación técnica
- **Checkpoint:** Usuario revisa antes de continuar

4. **Fase 3 - Validación (DeepSeek-R1-14B):**

- Identifica inconsistencias
- Detecta errores potenciales
- Sugiere mejoras
- Revisa consideraciones de seguridad
- Propone optimizaciones
- Señala elementos faltantes

5. **Output:**

- Documento MVP para clientes
 - Documentación técnica completa
 - Reporte de validación
 - Plan de desarrollo del sistema definitivo
-

Backend IA

- **Plataforma:** Ollama (local)
- **API:** HTTP REST (OpenAI-compatible)
- **Host:** localhost:11434
- **Alternativa futura:** vLLM (si se requiere más throughput)

Aplicación Principal

- **Lenguaje:** Python 3.11+
- **Tipo:** CLI/Desktop App
- **Contenedor:** Docker
- **Gestión de dependencias:** Poetry o pip-tools

Bibliotecas Core

- `ollama` - Cliente Python para Ollama
- `pydantic` - Validación de datos y estado
- `pyyaml` - Configuración
- `rich` - CLI mejorada con colores/progreso

Infraestructura

- Docker + Docker Compose
- NVIDIA Container Toolkit (para GPU)

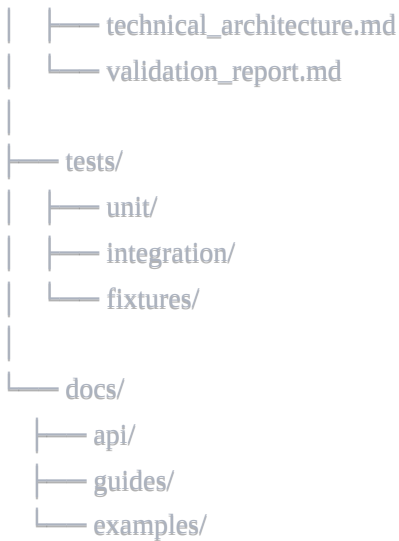
Frameworks NO utilizados (deliberadamente)

-  LangGraph - Demasiado complejo para flujo lineal
-  CrewAI - Overhead innecesario, problemas de logging
-  AutoGen - No aplicable a flujo no-conversacional

Estructura del Proyecto (Propuesta)

```
arqsysia/  
├── docker-compose.yml  
├── README.md  
└── FOUNDATIONAL_DOCUMENT.md # Este documento
```

```
graph TD; app[app/] --- Dockerfile[Dockerfile]; app --- requirements[requirements.txt]; app --- pyproject[pyproject.toml]; app --- arqsysia[arqsysia/]; arqsysia --- __init__py1[__init__.py]; arqsysia --- main[main.py]; arqsysia --- core[core/]; core --- __init__py2[__init__.py]; core --- orchestrator[orchestrator.py] --- P1[# Pipeline principal]; core --- state[state.py] --- P2[# ProjectState]; core --- models[models.py] --- P3[# Configuración modelos]; arqsysia --- phases[phases/]; phases --- __init__py3[__init__.py]; phases --- analyzer[analyzer.py] --- F1[# Fase 1]; phases --- codegen[codegen.py] --- F2[# Fase 2]; phases --- validator[validator.py] --- F3[# Fase 3]; arqsysia --- clients[clients/]; clients --- __init__py4[__init__.py]; clients --- ollama[ollama_client.py]; clients --- vllm[vllm_client.py] --- F4[# Futuro]; arqsysia --- outputs[outputs/]; outputs --- __init__py5[__init__.py]; outputs --- generator[generator.py] --- G[# Generador docs]; arqsysia --- utils[utils/]; utils --- __init__py6[__init__.py]; utils --- prompts[prompts.py]; utils --- validators[validators.py]; app --- config[config/]; config --- modelsyaml[models.yaml]; workspace[workspace/] --- U[# Entrada de usuario]; workspace --- requirements2[requirements/]; requirements2 --- example(example_project.md); output[output/] --- S[# Salida generada]; output --- mvp(mvp_proposal.md)
```

Principios de Diseño

1. Simplicidad sobre Complejidad

- Preferir soluciones directas sobre frameworks pesados
- Código legible es mejor que código "inteligente"
- No sobre-ingenierizar

2. Control Explícito

- Usuario tiene visibilidad total del proceso
- Checkpoints entre fases para aprobación manual
- Logging claro y detallado

3. Modularidad

- Cada fase es independiente y testeable
- Fácil reemplazar o mejorar componentes individuales
- Arquitectura evolutiva: puede migrar a agentes si es necesario

4. Open Source First

- Priorizar tecnologías open-source
- Modelos con licencias permisivas
- Sin vendor lock-in

5. Hardware-Conscious

- Diseñado para aprovechar la workstation disponible

- Uso eficiente de VRAM
 - No requiere infraestructura cloud
-



Roadmap de Desarrollo

Fase 0: Fundación ☒ (Completada)

- ☒ Definición de visión y arquitectura
- ☒ Selección de modelos
- ☒ Decisiones arquitecturales fundamentales
- ☒ Documento fundacional

Fase 1: Consultoría con DeepSeek-R1 (Próxima)

- ☐ Instalación de DeepSeek-R1-32B
- ☐ Script de consultoría para validar arquitectura
- ☐ Análisis experto del diseño de ArqSysIA
- ☐ Refinamiento del diseño basado en feedback del modelo

Fase 2: Infraestructura Base

- ☐ Configuración Docker + Docker Compose
- ☐ Cliente Ollama con retry logic
- ☐ State Manager básico
- ☐ Configuración de modelos (YAML)

Fase 3: Pipeline Orchestrator

- ☐ Implementación del orquestador principal
- ☐ Sistema de checkpoints
- ☐ Logging estructurado
- ☐ Manejo de errores robusto

Fase 4: Fases Individuales

- ☐ Implementación Analyzer (Fase 1)
- ☐ Implementación CodeGen (Fase 2)
- ☐ Implementación Validator (Fase 3)
- ☐ Generador de outputs

Fase 5: UX y Refinamiento

- ☐ CLI interactiva (Rich)

- ☐ Validación de inputs
- ☐ Templates de prompts optimizados
- ☐ Documentación de usuario

Fase 6: Testing y Estabilización

- ☐ Tests unitarios
- ☐ Tests de integración
- ☐ Casos de prueba reales
- ☐ Optimización de prompts

Fase 7: Features Avanzados (Futuro)

- ☐ Interfaz gráfica (opcional)
 - ☐ Exportación a múltiples formatos
 - ☐ Integración con herramientas de gestión de proyectos
 - ☐ Migración a vLLM si se requiere más throughput
 - ☐ Sistema de agentes (CrewAI) si se justifica
-



Métricas de Éxito

Objetivos Cuantitativos

- **Reducción de tiempo:** 60-80% menos tiempo en análisis arquitectural inicial
- **Calidad:** 90%+ de satisfacción con propuestas arquitecturales
- **Uso de recursos:** <14GB VRAM en operación normal
- **Velocidad:** <5 minutos para análisis completo de proyecto mediano

Objetivos Cualitativos

- Arquitecturas coherentes y bien justificadas
 - Código generado siguiendo mejores prácticas
 - Documentación clara y útil para clientes
 - Experiencia de usuario fluida e intuitiva
-



Consideraciones de Seguridad

Aislamiento

- Aplicación en Docker para aislamiento del host

- No requiere permisos elevados
- Acceso controlado a filesystem

Datos

- Todo procesamiento es local (no sale de la máquina)
- Sin envío de datos a APIs externas
- Información de clientes permanece privada

Modelos

- Licencias verificadas y compatibles con uso comercial
 - Modelos open-source auditables
 - Sin dependencia de servicios propietarios
-



Referencias y Recursos

Modelos LLM

- **DeepSeek-R1:** <https://github.com/deepseek-ai/DeepSeek-R1>
- **Qwen2.5-Coder:** <https://ollama.com/library/qwen2.5-coder>
- **Ollama Library:** <https://ollama.com/library>

Frameworks Considerados (pero no usados)

- **LangGraph:** <https://github.com/langchain-ai/langgraph>
- **CrewAI:** <https://github.com/joaomdmoura/crewAI>
- **AutoGen:** <https://github.com/microsoft/autogen>

Plataformas de Inferencia

- **Ollama:** <https://ollama.com>
- **vLLM:** <https://github.com/vllm-project/vllm>
- **llama.cpp:** <https://github.com/ggerganov/llama.cpp>

Hardware

- **NVIDIA RTX 5070 Ti:** <https://www.nvidia.com/es-la/geforce/graphics-cards/50-series/>
-



Notas Finales

Para Futuros Claudes

Este documento representa las **decisiones arquitecturales fundamentales** del proyecto ArqSysIA. Antes de sugerir cambios significativos en la arquitectura, considera:

1. **Las razones documentadas** para cada decisión
2. **El contexto del usuario:** workstation individual, no necesita alta concurrencia
3. **El principio de simplicidad:** no sobre-ingenierizar
4. **La arquitectura evolutiva:** puede migrar a agentes si se justifica

Filosofía del Proyecto

"La mejor arquitectura no es la más sofisticada, sino la más apropiada para el problema específico. ArqSysIA prioriza control, simplicidad y efectividad sobre complejidad innecesaria."

Contacto

- **Ubicación:** Crespo, Entre Ríos, Argentina
- **Fecha inicio:** Octubre 3, 2025
- **Estado:** Fase de diseño completada, próximo paso: implementación



Control de Versiones del Documento

Versión	Fecha	Cambios
1.0	2025-10-03	Documento fundacional inicial

Fin del Documento Fundacional

Este documento debe mantenerse actualizado conforme el proyecto evoluciona, pero las decisiones fundamentales aquí documentadas representan la intención y diseño original.